

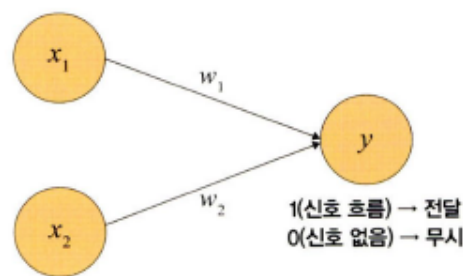
Week2_예습과제_진예원

4장 딥러닝 시작

4.1 인공 신경망의 한계와 딥러닝 출현

퍼셉트론

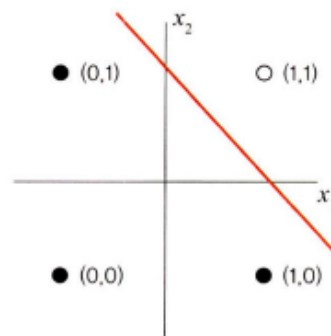
- 오늘날 인공 신경망에서 이용하는 구조(입력층, 출력층, 가중치로 구성된 구조)
- 프랭크 로젠블라트가 1957년에 고안한 선형 분류기
- 오늘날 신경망(딥러닝)의 기원이 되는 알고리즘
- 다수의 신호(흐름이 있는)를 입력으로 받아 하나의 신호를 출력함. 이 신호를 입력으로 받아 '흐른다/안 흐른다(1 or 0)'는 정보를 앞으로 전달하는 원리로 작동함.



논리 게이트(입력 두 개 있다고 가정)

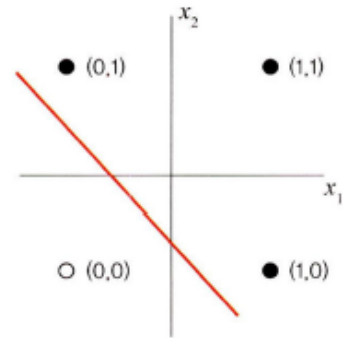
1. AND 게이트: 모든 입력이 1일때 작동. 어떤 하나라도 0이면 작동 멈춤

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1



2. OR 게이트: 입력에서 둘 중 하나만 1이거나 둘 다 1일때 작동. 모두가 0을 갖는 경우 제외 나머지 모두 1 값을 가짐

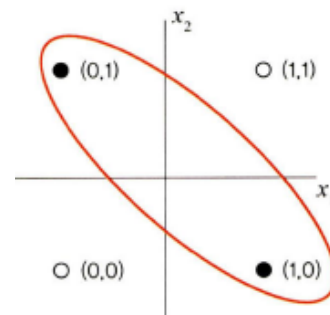
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1



3. XOR 게이트: 배타적 논리합. 입력 두 개 중 한 개만 1일때 작동

- 데이터가 비선형적으로 분리되므로 제대로 된 분류 hard → 단층 퍼셉트론에선 학습 불가능(AND, OR에 대해서는 가능)
- 방안 : 입력층과 출력층 사이에 하나 이상의 중간층(은닉층)을 두어 비선형적으로 분리되는 데이터에 대해서도 학습 가능하도록 다층 퍼셉트론 고안
 * 입력층과 출력층 사이에 은닉층 여러 개 있는 신경망 = 심층 신경망(DNN) = 딥러닝

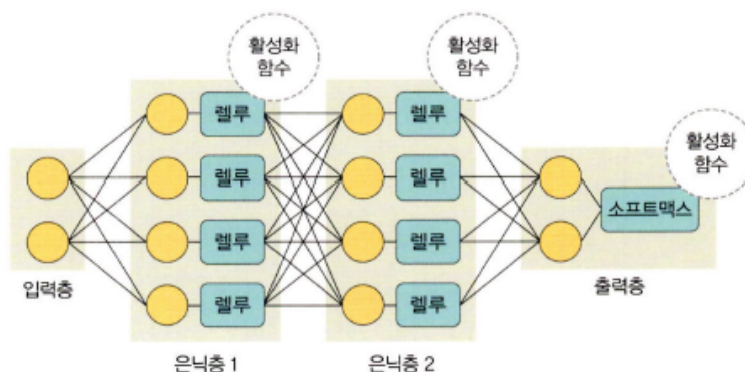
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



4.2 딥러닝 구조

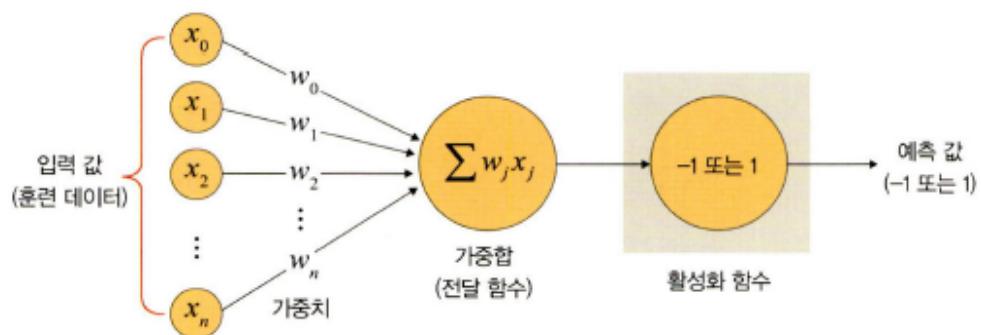
4.2.1 딥러닝 용어

구조 - 입력층, 출력층, 두 개 이상의 은닉층

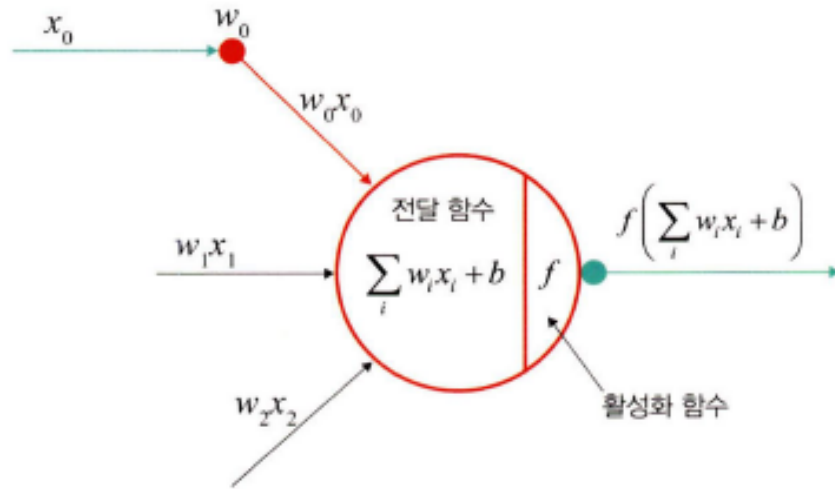


구성 요소

1. 입력층: 데이터를 받아들임
2. 은닉층: 모든 입력 노드부터 입력 값을 받아 가중합 계산 → 이 값을 활성화 함수에 적용하여 출력층에 전달
3. 출력층: 신경망의 최종 결괏값이 포함된 층
4. 가중치: 노드와 노드 간 연결 강도
 - ex) w_1 값이 0 또는 0.001(0 가까이)라면 → x_1 이 아무리 큰 값이라도 $x_1 \times w_1$ 값은 0이거나 가까운 값이 됨



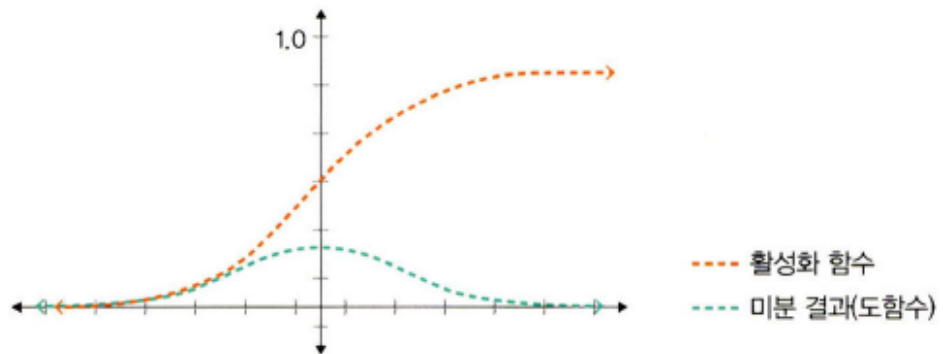
5. 바이어스: 가중합에 더해 주는 상수. 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절
6. 가중합 = 전달 함수: 가중치와 신호의 곱을 합한 것
 - 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달됨. 이 값들을 모두 더함
 - 노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기에 전달함수라고도 함
 - $\sum w_i x_i + b$ (w : 가중치, b : 바이어스)



7. 활성화 함: 전달 함수에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수

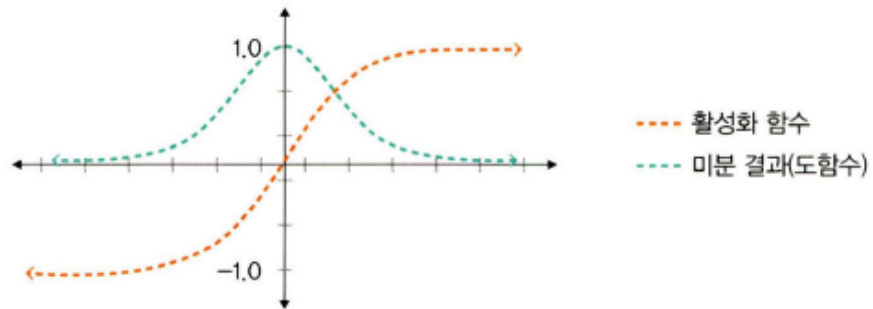
a. 시그모이드 함수 : 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형

- 주로 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현할 때 사용
- 기울기 소멸 문제 O
- $f(x) = 1/(1 + e^{-x})$



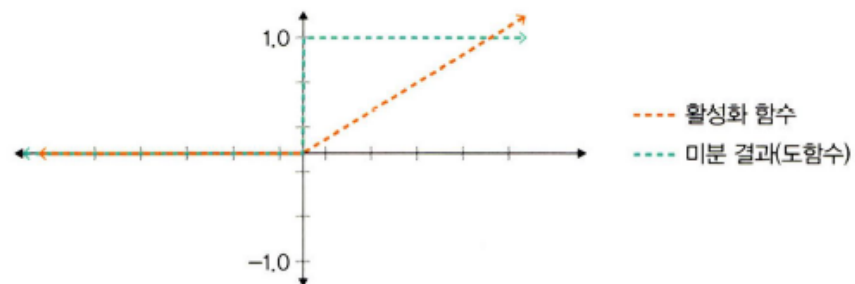
b. 하이퍼볼릭 탄젠트 함수 : -1 ~ 1 사이의 비선형 형태로 변형

- 시그모이드(a)에서 결괏값이 평균이 0이 아닌 양수로 편향되는 문제 보완
- 기울기 소멸 문제 O



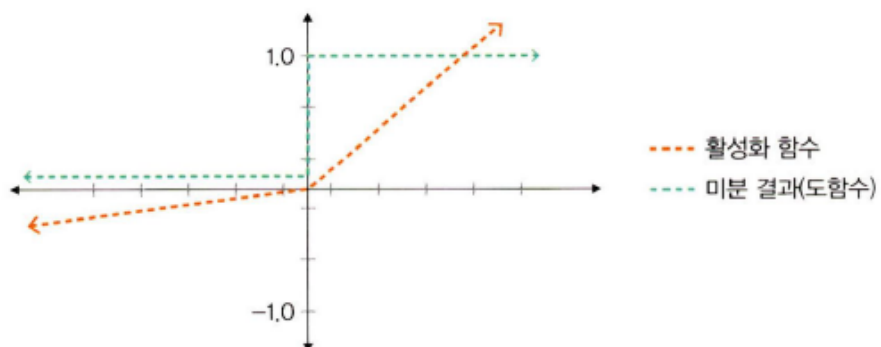
c. 렐루 함수 : 입력이 음수 $\rightarrow 0$, 양수 $\rightarrow x$ 출력

- 경사 하강법에 영향 $X \rightarrow$ 학습 속도 $\uparrow$
- 기울기 소멸 문제 X
- 보통 은닉층에서 사용
- 하이퍼볼릭 탄젠트 함수 대비 학습 속도 6배 빠름
- 음수 값 입력받으면 항상 0 출력하여 학습 능력 감소 문제
 $\Rightarrow$ 방안: 리키 렐루 함수



d. 리키 렐루 함수: 입력이 음수 $\rightarrow 0$ 이 아닌 0.001처럼 매우 작은 수 반환

- 입력 값이 수렴하는 구간 제거되어 렐루 함수의 문제 해결



e. 소프트맥스 함수: 입력 0~1 출력되도록 정규화하여 출력 값 총합이 1이 되도록 함

- 보통 딥러닝에서 출력 노드의 활성화 함수로 사용
- $y_k = \exp(a_k) / \sum \exp(a_i)$ ($\exp(x)$: 지수 함수, n : 출력층 뉴런 개수, y_k : 그 중 k번째 출력)

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

8. 손실 함수: 가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

- 학습률과 손실함수의 순간 기울기를 이용하여 가중치 업데이트. 미분의 기울기 이용하여 오차 비교, 최소화하는 방향으로 이동
- 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표. 값 ↑ → 많이 틀렸다/0에 가까우면 완벽하게 추정할 수 있다는 의미

a. 평균 제곱 오차: 실제 값과 예측 값의 차이(error)를 제곱하여 평균을 냄

- 실제 값과 예측 값의 차이 ↑ → 평균 제곱 오차 값 ↑ ↔ 값 ↓ → 예측력 ↑
- 주로 회귀에서 손실 함수로 사용

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$

b. 크로스 엔트로피 오차

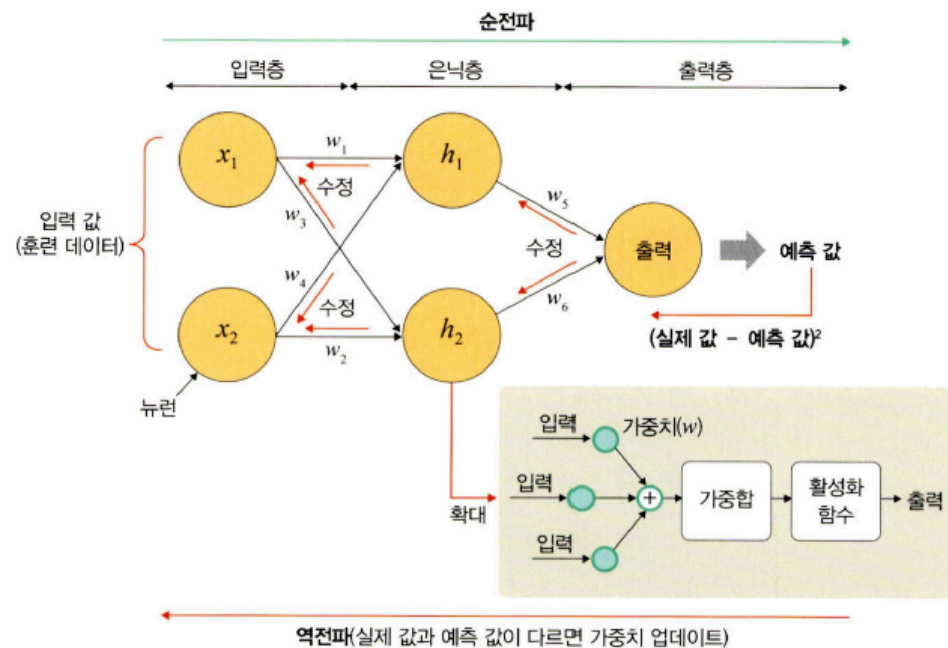
- 분류 문제에서 원-핫 인코딩 했을 때만 사용할 수 있는 오차계산법
- 보통 분류 문제에서 시그모이드 함수 사용. 시그모이드 함수의 자연 상수 e 때문에 평균 제곱 오차 적용 → 매끄럽지 못한(울퉁불퉁한) 그래프 출력
⇒ 크로스 엔트로피 손실 함수 사용하면 경사 하강법 과정에서 학습이 지역 최소점에서 멈출 수 O
- 자연 상수 e에 반대되는 자연 로그를 모델의 출력 값에 취함

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 y_i : 정답 레이블
 i : 데이터의 차원 개수

4.2.2 딥러닝 학습

딥러닝 학습 = 순전파 + 역전파



순전파(feedforward)

- 네트워크에 훈련 데이터 들어올 때 발생. 데이터를 기반으로 예측 값 계산 위해 전체 신경망을 교차해 지나감
- 모든 뉴런이 이전 층 뉴런에서 수신한 정보에 변환(가중합, 활성화 함수)을 적용하여 다음 층(은닉층)의 뉴런으로 전송하는 방식
- 네트워크 통해 입력 데이터 전달
- 데이터가 모든 층 통과하고 모든 뉴런 계산 완료하면 그 예측 값은 최종 층(출력층)에 도달

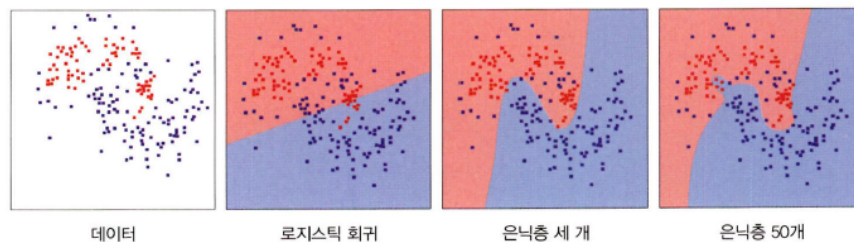
역전파(backpropagation)

- 그다음 손실 함수로 네트워크의 예측 값과 실제 값의 차이(손실, 오차) 측정

- 손실 함수 비용이 0(이상적 상태)에 가깝도록 하기 위해 모델 훈련 반복하며 가중치 조정
- 손실(오차) 계산되면 그 정보는 역으로 전파됨(출력층 → 은닉층 → 입력층) = 역전파
- 출력층에서 시작된 손실 비용은 은닉층의 모든 뉴런으로 전파 But 은닉층 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도에 따라(가중치) 값이 달라짐
= 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 뺌
- 이 과정을 출력층 → 은닉층 → 입력층 순서로 모든 뉴런에 대해 진행하여 계산된 각 뉴런 결과를 또다시 순전파의 가중치 값으로 사용

4.2.3 딥러닝의 문제점과 해결 방안

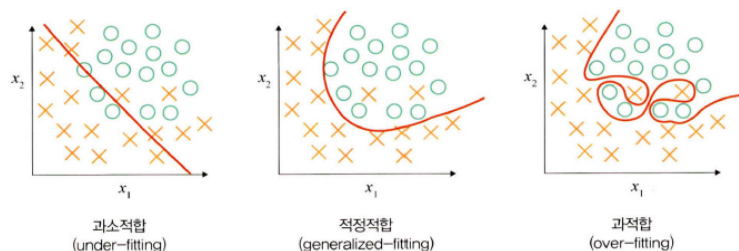
- 핵심: 활성화 함수가 적용된 여러 은닉층 결합하여 비선형 영역을 표현하는 것



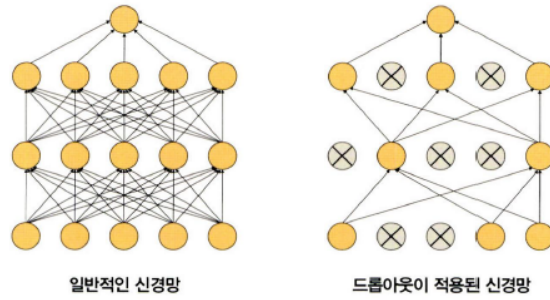
- 활성화 함수가 적용된 은닉층 개수 ↑ → 데이터 분류 잘됨
But 은닉층 많을수록 다음 문제점 발생

과적합 문제 발생

- 훈련 데이터 과하게 학습하여 발생
- 보통 훈련 데이터는 실제 데이터의 일부분이므로, 훈련 데이터 과하게 학습하면 오차는 ↓ But 검증 데이터에 대해서는 오차 ↑

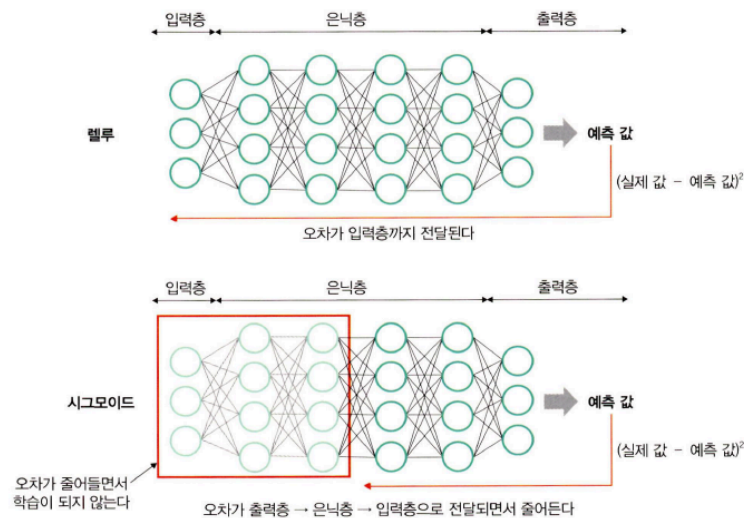


- 해결방안: 드롭아웃(dropout) : 학습 과정 중 임의로 일부 노드들 학습에서 제외시킴



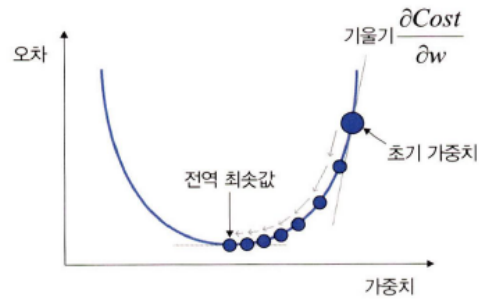
기울기 소멸 문제 발생

- 은닉층 많은 신경망에서 주로 발생
- 출력층에서 은닉층으로 전달된 $\hookrightarrow$ 오차가 크게 줄어들어 학습이 되지 않는 현상
- 기울기 소멸로 학습되는 양이 0에 가까워져 학습이 더디게 진행된다 오차 더 줄이지 못하고 그 상태로 수렴
- 해결방안: 시그모이드, 하이퍼볼릭 탄젠트 대신 렐루 활성화 함수 사용

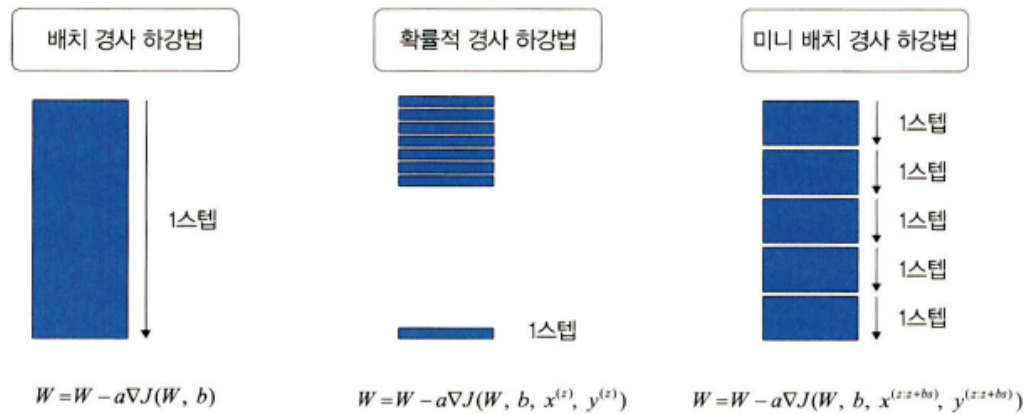


성능이 나빠지는 문제 발생

- 경사 하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정 반복하는데, 이때 성능 나빠지는 문제 발생



- 해결방안: 확률적 경사 하강법, 미니 배치 경사 하강법



0. 배치 경사 하강법(BGD): 전체 데이터셋에 대한 오류 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트함

- 전체 훈련 데이터셋에 대해 가중치를 편미분함
- 한 스텝에 모든 훈련 데이터셋 사용하여 학습 오래 걸림
⇒ 방안: 확률적 경사 하강법

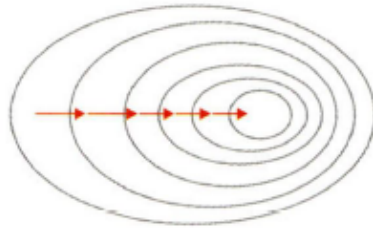
손실 함수의 값을 최소화하기 위해 기울기(∇) 이용

$$W = W - a \nabla J(W, b)$$

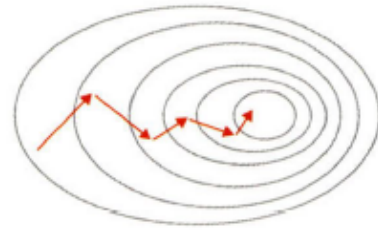
(a : 학습률, J : 손실 함수)

1. 확률적 경사 하강법(SGD): 임의로 선택한 데이터에 대해 기울기 계산.

- 적은 데이터 사용하므로 계산 빠름
- 파라미터 변경 폭이 불안정, 때론 배치 경사 하강법보다 정확도 ↓
- 속도가 빠름



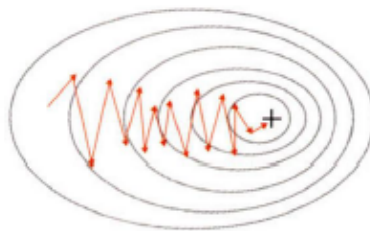
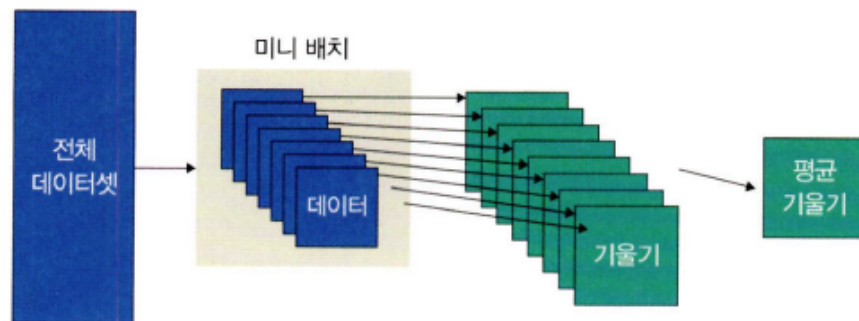
배치 경사 하강법



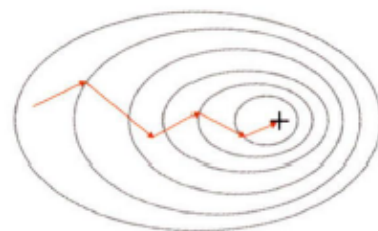
확률적 경사 하강법

2. 미니 배치 경사 하강법: 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델 업데이트해서 학습

- 전체 데이터 계산하는 것보다 빠름
- 확률적 경사 하강법보다 안정



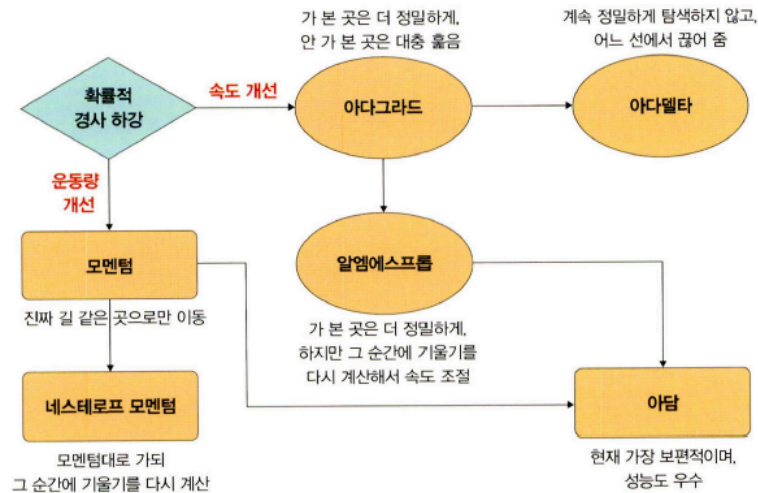
확률적 경사 하강법



미니 배치 경사 하강법

옵티마이저

- 확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제 해결 위해 학습 속도와 운동량 조절하는 옵티마이저 적용해볼 수 있다.



• 속도를 조정하는 방법

1. 아다그리드: 변수(가중치)의 업데이트 횟수에 따라 학습률 조정하는 방법

- 많이 변화하지 않는 변수들의 학습률은 크게, 많이 변화하는 변수들의 학습률은 작게 함
- 많이 변화한 변수는 최적 값에 근접했을 것이라는 가정하에 작은 크기로 이동하면서 세밀하게 값 조정하고, 반대로 적게 변화한 변수들은 학습률을 크게 하여 빠르게 오차 값을 줄이고자 함
- 파라미터마다 다른 학습률을 주기 위해 G 함수 추가. G: 이전 G 값의 누적(기울기 크기의 누적). 기울기 $\uparrow \rightarrow G \uparrow$ 학습률(η)은 작아짐
- 파라미터가 많이 학습되었으면 작은 학습률로 업데이트, 덜 학습되었으면 개선의 여지가 많으므로 높은 학습률로 업데이트됨
- 기울기 0에 수렴하는 문제로 사용 $X \Rightarrow$ 알엠에스프롭

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

2. 아다델타:

- 아다그리드에서 $G \uparrow \rightarrow$ 학습 멈추는 문제 해결
- 아다그리드 수식에서 학습률(η)을 D함수(가중치의 변화량 크기를 누적한 값)로 변환했기 때문에 학습률에 대한 하이퍼파라미터 필요 X

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\varepsilon}}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2$$

3. 알엠에스프롭

- 아다그리드의 $G(i)$ 값이 무한히 커지는 것을 방지
- 아다그리드에서 학습 안 되는 문제 해결 위해 G 함수에서 Γ 만 추가
- 사용자가 Γ 값을 이용하여 학습률 크기를 비율로 조정 가능

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

• 운동량 조정하는 방법

1. 모멘텀: 경사 하강법과 마찬가지로 매번 기울기를 구하지만, 가중치를 수정하기 전에 이전 수정 방향(+,-)을 참고하여 같은 방향으로 일정한 비율만 수정하는 방법

- 수정이 양(+) 방향과 음(-)의 방향으로 순차적으로 일어나는 지그재그 현상 줄어든다
- 이전 이동 값 고려하여 일정 비율만큼 다음 값을 결정하므로 관성 효과 얻을 수 있다
- SGD(확률적 경사 하강법)와 함께 사용
- 기울기 크기와 반대 방향만큼 가중치 업데이트. 기울기가 크면 아래쪽(-) 방향으로 업데이트

$$w(i+1) = w(i) - \eta \nabla E(w(i))$$

- SGD 모멘텀은 SGD에서 기울기를 속도로 대체 → 일정 속도의 일정 부분 반영. 이전에 학습했던 속도와 현재 기울기를 반영해서 가중치 구함

$$w(i+1) = w(i) - v(i)$$

$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i))$$

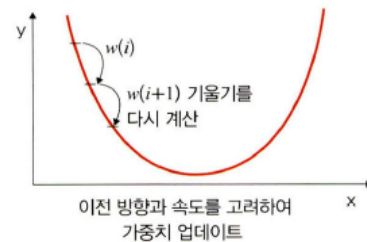
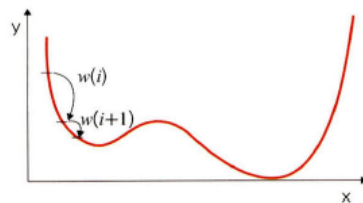
- 모멘텀 값은 0.9에서 시작하며 0.95, 0.99처럼 조금씩 증가시키며 사용

2. 네스테로프 모멘텀: 모멘텀 값과 기울기 값이 더해서 실제 값을 만드는 기존 모멘텀과 달리 모멘텀 값이 적용된 지점에서 기울기 값 계산

- 모멘텀 방법은 멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있는 단점
⇒ 네스테로프 방법은 모멘텀으로 절반 정도 이동한 후 어떤 방식으로 이동해야 하는지 다시 계산하여 결정
- 모멘텀의 장점인 빠른 이동 속도는 가져가며 멈추어야 할 적절한 시점에서 제동을 거는 데 훨씬 용이
- 속도 구하는 과정에서 차이. 이전에 학습했던 속도와 현재 기울기에서 이전 속도를 뺀 변화량을 반영해서(더해서) 가중치를 구함

$$w(i+1) = w(i) - v(i)$$

$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))$$



• 속도와 운동량에 대한 혼용 방법

1. 아담: 모멘텀 + 알엠에스프롭. 알엠에스프롭 특징인 기울기의 제곱을 지수 평균한 값과 모멘텀 특징인 $v(i)$ 를 수식의 활용하여 가중치를 업데이트함

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1 - \gamma_2) (\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

4.2.4 딥러닝을 사용할 때 이점

특성 추출

- 컴퓨터가 입력받은 데이터 분석하여 일정한 패턴이나 규칙 찾아내려면 사람이 인지하는 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환해 주어야 함. 이때 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업을 특성 추출이라고 함
- 딥러닝 이전 머신 러닝 알고리즘(SVM, 나이브 베이즈, 로지스틱 회귀)의 특성 추출은 복잡, 수집된 데이터에 대한 전문 지식이 필요했음
⇒ 딥러닝에서는 이러한 특성 추출 과정을 알고리즘에 통합시킴
∴ 데이터 특성 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조

빅데이터의 효율적 활용

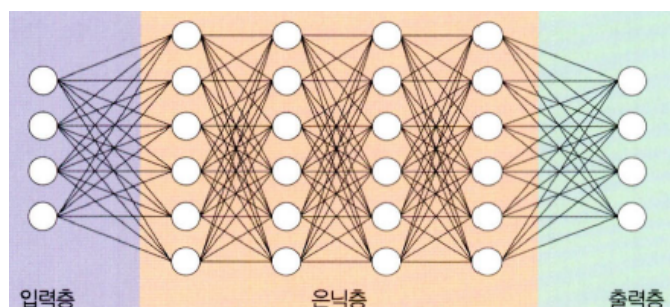
- 특성 추출을 알고리즘에 통합시킬 수 있던 이유는 빅데이터 때문. 딥러닝 학습을 이용한 특성 추출은 데이터 사례 ↑ 수록 성능 ↑ 때문
- if 확보 데이터 ↓ → 딥러닝 성능 향상 기대 hard ⇒ 머신 러닝 고려 필요

4.3 딥러닝 알고리즘

- 딥러닝 알고리즘은 심층 신경망을 사용한다는 공통점을 가짐. 목적에 따라 합성곱 신경망(CNN), 순환 신경망(RNN), 제한된 볼츠만 머신(RBM), 심층 신뢰 신경망(DBN)으로 분류됨
- 딥러닝에서 많이 사용되는 알고리즘은 CNN, RNN

4.3.1 심층 신경망(DNN)

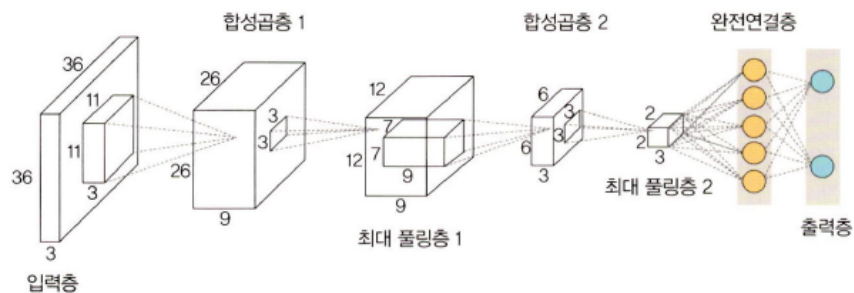
- 입력층과 출력층 사이에 다수의 은닉층 포함하는 인공 신경망
- 다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류 가능 (머신러닝에서는 비선형 분류 위해 여러 트릭을 사용했음)
- 학습을 위한 연산량 ↑
- 기울기 소멸 문제
- 해결방안: 드롭아웃, 렐루 함수, 배치 정규화 적용 필요



4.3.2 합성곱 신경망(CNN) → 5, 6장

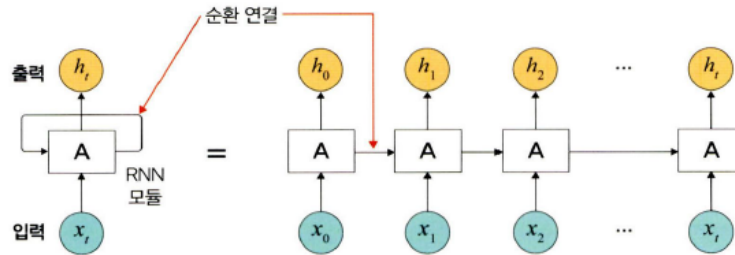
- 합성곱층(convolutional layer)과 풀링층(pooling layer)을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘
- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용한 신경망
- 이미지에서 객체, 얼굴, 장면을 인식하기 위해 패턴 찾는데 특히 유용
- ex) LeNet-5, AlexNet / (층을 더 깊게 쌓은 신경망) VGG, GoogLeNet, ResNet

- 기존 신경망과의 차별성
 - 각 층의 입출력 형상 유지
 - 이미지 공간 정보 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
 - 복수 필터로 이미지의 특징 추출, 학습
 - 추출한 이미지의 특징 모으고 강화하는 풀링층 존재
 - 필터를 공유 파라미터로 사용하기 때문에 일반 인공 신경망과 비교하여 학습 파라미터가 매우 적음



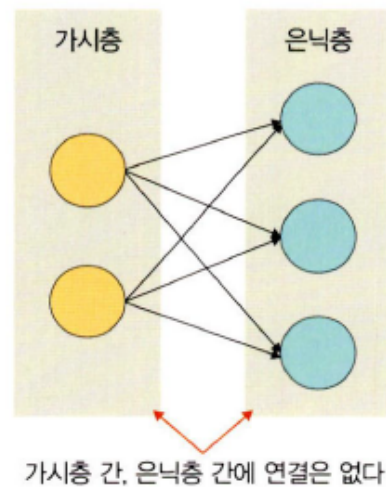
4.3.3 순환 신경망(RNN) → 7장

- 시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
- 순환 신경망의 순환(recurrent): 자기 자신을 참조한다 = 현재 결과가 이전 결과와 연관이 있다
- 특징
 - 시간성을 가진 데이터가 많음
 - 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
 - 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적
 - 매우 긴 데이터를 처리하는 연구가 활발히 진행됨
- 기울기 소멸 문제
 - ⇒ 해결방안: 메모리 개념 도입한 LSTM 많이 사용됨
- 자연어 처리 분야와 자주 사용됨
 - ex) 언어 모델링, 텍스트 생성, 자동 번역(기계 번역), 음성 인식, 이미지 캡션 생성



4.3.4 제한된 볼츠만 머신(RBM)

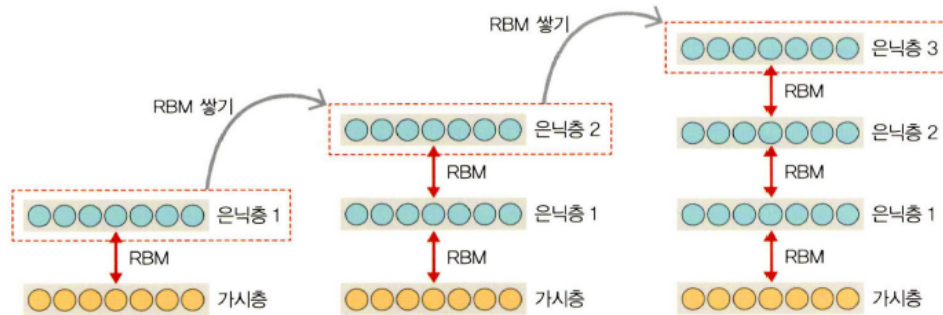
- 가시층(visible layer)와 은닉층(hidden layer)으로 구성된 모델
- 이 모델에서 가시층은 은닉층과만 연결됨(가시층과 가시층, 은닉층과 은닉층 사이에 연결 X)
- 특징
 - 차원 감소, 분류, 선형 회귀 분석, 현업 필터링, 특성 값 학습, 주제 모델링에 사용
 - 기울기 소멸 문제 해결 위해 사전 학습 용도로 활용 가능
 - 심층 신뢰 신경망(DBN)의 요소로 활용



4.3.5 심층 신뢰 신경망(DBN)

- 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망. 사전 훈련된 볼츠만 머신을 층층이 쌓아 올린 구조
- 레이블 없는 데이터에 대한 비지도 학습 가능
- 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정 구현할 때 유용
- 학습 절차
 1. 가시층과 은닉층 1에 제한된 볼츠만 머신을 사전 훈련한다.

2. 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 제한된 볼츠만 머신을 사전 훈련한다.
3. 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN을 완성한다.



• 특징

- 순차적으로 심층 신회 신경망을 학습시켜 가면서 계층적 구조를 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성 추출
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용